

Photon Camera HDR v4 — v2

Complete Pipeline Reverse-Engineering Analysis (v2 — with dev notes & proposed fixes)

Package	com.particlesdevs.photoncamera.hdr
Version	0.97 (build 26112)
Target SDK	35 (Android 15)
Min SDK	26 (Android 8)
Decompiler	JADX 1.5.1 + Apktool 2.10.0
Analysis Date	2026-07-25 (Updated with dev notes)

This document is a complete reverse-engineering analysis of the Photon Camera HDR v4 Android application. It covers the entire image processing pipeline from raw Bayer frame capture through multi-frame merging, local tone mapping, denoising, demosaicing, color correction, global tone mapping, and sharpening — down to individual GLSL shader logic and tunable parameters.

Table of Contents

1. Architecture Overview
2. Stage 1: HDR Frame Merging (PyramidMerging)
3. Stage 2: PostPipeline — Node Chain
4. Exposure Fusion & Local Tone Mapping
5. Denoising — ESD3D, SmartNR, Bilateral
6. Noise Model
7. Demosaicing
8. Color Processing Pipeline
9. Histogram Equalization & Global Tone Mapping
10. Sharpening
11. Key GLSL Shader Reference
12. Tunable Parameter Reference
13. Adjustment Strategy Guide
14. Recompilation Guide

1. Architecture Overview

The entire pipeline is **OpenGL ES 3.0 compute shader** based, operating on GPU textures. All intermediate processing happens in **float16** (half-precision) or **float32** (single-precision) — no 8-bit quantization until final JPEG output. The architecture follows a **node-chain pattern** where each processing stage is a self-contained Node that reads from the previous node's output texture and writes to a new texture.

1.1 High-Level Data Flow

Stage	Node	Description	Exec
Stage 1	PyramidMerging	N raw Bayer frames → 1 merged raw	GPU compute
	Bayer2Float	Raw uint16 → normalized float	GPU fragment
	ABLC	Black level correction	GPU fragment
	HotPixelFilter	Statistical hot pixel removal	GPU compute
	Initial	Initial color space setup	GPU fragment
Stage 2	ExposureFusionBayer2	LTM — local tone mapping	GPU fragment+pyramid
	Demosaic3	Bayer → RGB demosaicing	GPU fragment
	AWB	Auto white balance	GPU fragment
	ChromaticFlow	Chromatic aberration correction	GPU fragment
	ColorD	Color correction matrix	GPU fragment
	ESD3D	Edge-preserving denoise (luma)	GPU fragment
	SmartNR	Bilateral guide denoise (INACTIVE)	GPU fragment
	Bilateral / BilateralSep	Additional bilateral (INACTIVE)	GPU fragment
	Wavelet	Wavelet-domain processing (INACTIVE)	GPU compute
	LensCorrection	Lens distortion / vignetting	GPU fragment
	Equalization	Histogram EQ + tonemap (INACTIVE)	GPU fragment
	GlobalToneMapping	Global tone mapping (INACTIVE)	GPU fragment
	Sharpen / Sharpen2	Luminance sharpening	GPU fragment
	RotateWatermark	Rotation + watermark	GPU fragment

1.2 Key Source Packages

Package	Purpose
processing.opengl.postpipeline	PostPipeline + all post-processing Nodes
processing.opengl.scripts	PyramidMerging, AverageRaw, GLHistogram, etc.
processing.opengl	GLBasePipeline, GLProg, GLTexture, GLUtils
processing.render	Parameters, NoiseModeler, ColorCorrectionTransform
processing.parameters	IsoExpoSelector, FrameNumberSelector
processing.processor	HdrxProcessor (main HDR orchestrator)
capture	CaptureController (Camera2 API frame acquisition)
api	Settings, Camera2ApiAutoFix, NativeEngine

2. Stage 1: HDR Frame Merging (PyramidMerging)

Before the post-processing pipeline runs, multiple raw frames are captured at varying exposure/ISO settings and merged into a single high-dynamic-range raw buffer. This is handled by **HdrxProcessor** which orchestrates **PyramidMerging**.

2.1 Frame Capture & Classification

Frames are captured via Camera2 API with manual exposure control. The **IsoExpoSelector** determines ISO and exposure time for each frame, classifying each as one of three exposure layers:

- **Low** — underexposed (shadows preserved, highlights may clip)
- **Normal** — metered exposure
- **High** — overexposed (highlights preserved, shadows noisy)

The bracketing ratio is configurable (1×, 4×, or 8×). For HDR mode, frames are captured in a repeating pattern: every 3rd frame gets the high-exposure boost, while the others remain at normal. The exposure multiplier is applied by adjusting ISO and exposure time while respecting sensor limits.

2.2 Gyro-Based Frame Selection

Each frame is associated with a **GyroBurst** containing gyroscope data during the capture. The gyro data produces a blur score (lower = sharper). Frames are sorted by this score, and the sharpest frame is preferred as the reference. If more than 10 frames are captured, the blurriest 25% are discarded (unless removing them would lose all high-exposure frames).

2.3 Hot Pixel Detection & Correction

Before merging, the pipeline detects and corrects hot pixels using a statistical approach:

- All frames are averaged at half resolution into a single reference image
- For each pixel, a 3×3 median is computed across the Bayer quad (R, G1, G2, B)
- Difference between pixel and median is compared against noise model: $\text{diff} > \text{noise} \times \text{detectThr}$
- Detected hot pixels are ranked by z-score; if count exceeds MAX_REASONABLE_HOTPIXELS (2000), weakest are pruned
- Statistical filtering removes detections below $\text{mean} - 1.5 \times \text{stddev}$
- Corrected by replacing with the averaged frame value

Parameter	Default	Range	Description
detectThr	1.5	0.5–5.0	Higher = fewer hot pixels detected
MAX_HOT_PIXELS	65535	16384–262144	SSBO buffer limit
MAX_REASONABLE_HOTPIXELS	2000	1000–10000	Pruning threshold
enableHotPixelCorrection	false	bool	Enable correction pass
enableAdaptiveNoise	true	bool	Use stdev-based noise model
noiseMpyHigh	3.0	1.0–4.0	Adaptive noise high multiplier
noiseMpyLow	0.333	0.0–1.0	Adaptive noise low multiplier

2.4 Pyramid Merging Algorithm

The core merging uses a multi-scale Gaussian pyramid approach. For each frame:

- The raw Bayer data is normalized: `bayer = (raw - blackLevel) / (whiteLevel - blackLevel)`
- Exposure compensation is applied: `bayer *= 1.0 / referenceFrame.exposureMultiplier`
- A running average is maintained: `output = mix(current, new, 1/frameCount)`
- After averaging, hot pixel detection runs on the merged result
- The merged raw is then rescaled: black levels are adjusted proportionally to the new white level of 65535

2.5 GLSL Shader: merge00.glsl

This compute shader normalizes each Bayer quad (2x2 tile) from uint16 to float [0,1], applies exposure compensation, and optionally adds noise-shaped dithering:

```
vec4 bayer = getBayerVec(xy*TILE, inTexture); // Read 2x2 Bayer quad
bayer = (bayer - blackLevel) / (whiteLevel - blackLevel); // Normalize
bayer *= exposure; // Exposure compensation
imageStore(outTexture, xy, bayer);
```

2.6 GLSL Shader: avermix.glsl

Simple running average via linear interpolation:

```
vec4 current = imageLoad(currentTexture, coord);
vec4 newVal = imageLoad(newTexture, coord);
imageStore(outTexture, coord, mix(current, newVal, weight)); // weight = 1/frameCount
```

3. Stage 2: PostPipeline — Node Chain

After frame merging, the merged raw buffer enters the **PostPipeline**, a sequential chain of ~25 processing nodes. Each node inherits from **Node** and implements a `run()` method that reads from the previous node's output texture and writes to a new one. The pipeline is managed by **GLBasePipeline** which handles texture lifecycle and double-buffering.

3.1 Node Execution Model

- Each node has access to: `previousNode.outputTexture`, the GLSL program, utility functions, parameters, and settings
- Nodes can create intermediate textures (freed after use)
- Double-buffered textures (`f10716f` / `f10717g`) alternate as scratch space
- The `@Tunable` annotation exposes parameters to the tuning system
- Runtime overrides via `PhotonCameraTuning.ini` file

3.2 Complete Node Sequence

#	Node	Description
1	Bayer2Float	Converts uint16 raw to normalized float. Applies gain map (lens shading correction), black level subtraction, and white balance analog gains. Outputs to float16 texture.
2	ABLC	Additional black level correction — subtracts residual black level per channel (R, G1, G2, B).
3	HotPixelFilter	Second-pass hot pixel removal using statistical analysis on the float-converted data.
4	ImpulsePixelFilter	Impulse noise removal — detects and replaces isolated outlier pixels.
5	Initial	Initial processing — applies precorrection and sets up color space foundations.
6	ExposureFusionBayer2	Local Tone Mapping — the core HDR rendering step. Creates exposure pyramid and fuses. See Section 4.
7	ExposureFusionBayer3	Optional second fusion pass for additional dynamic range compression.
8	Demosaic3	Bayer demosaicing — interpolates R, G, B from the Bayer pattern. Uses directional interpolation with edge awareness.
9	AWB	Auto White Balance — applies per-channel white balance gains computed from the scene.
10	ChromaticFlow	Chromatic aberration correction — compensates for lateral CA using gradient-based flow.
11	CorrectingFlow	Additional geometric/color correction pass.
12	ColorD	Color correction — applies the Camera2 CCM (Color Correction Matrix) and adapts based on scene analysis.
13	ESD3D	Edge-Preserving 3D Denoise — luma denoising using hybrid SNN bilateral. See Section 5.
14	ESD3D2	Second ESD3D pass with potentially different parameters.
15	SmartNR	Bilateral guide denoise — separates luma and chroma denoising. See Section 5.
16	Bilateral	Standard 2D bilateral filter with noise-model-driven sigma.
17	BilateralSeparable	Separable bilateral filter (vertical + horizontal) for efficient additional smoothing.
18	Wavelet	Wavelet-domain processing — multi-scale decomposition, thresholding, and reconstruction.
19	LensCorrection	Lens distortion and vignetting correction using camera intrinsics.
20	Equalization	Histogram equalization + polynomial tone mapping. See Section 9.
21	GlobalToneMapping	Two-pass global tone mapping using downsampled luminance guide. See Section 9.

#	Node	Description
22	Sharpen	Luminance-domain sharpening using unsharp mask.
23	Sharpen2	Second sharpening pass with different kernel.
24	CaptureSharpening	Capture-domain sharpening tuned to the capture parameters.
25	RotateWatermark	Final rotation (sensor orientation) and optional watermark overlay.

4. Exposure Fusion & Local Tone Mapping

Local Tone Mapping (LTM) is the most complex and impactful stage of the pipeline. It compresses the high dynamic range of the merged raw into a displayable range while preserving local contrast and detail. Photon Camera uses **Laplacian pyramid fusion** operating directly on Bayer data.

4.1 Exposure Fork Calculation

The pipeline first analyzes the image histogram to determine how much gain to apply to dark areas and how much to reduce bright areas:

- Histogram is computed on a 4× downsampled version of the brightness channel
- **Overexposure detection:** weighted average position of pixels above 50% brightness, using $weight = \max(1 - 1.6 \times pos, 0)$
- **Underexposure detection:** weighted average of pixels 50-94% brightness, with gamma-k correction
- **Over-exposure gain:** $max2 = 128 / (overexp_pos + 1)$ — boosts dark regions
- **Under-exposure reduction:** $min = 179.2 / (underexp_pos + 1)$ — pulls back highlights
- **Noise clamp:** $max2 = \min(max2, noiseMax / \sqrt{noiseS \times 0.5 + noise0})$ — prevents boosting noise
- Final: $overExpo = \text{clamp}(max2 \times overExposeMpy, fusionExpoLowLimit, fusionExpoHighLimit)$

4.2 Spline Tone Curve

A 5-point cubic spline defines the tone curve mapping input luminance to output. Default control points:

Input	Output	Shadow Weight	Region
0.00	0.00	8.0	Deep shadows
0.07	0.07	4.0	Shadows
0.20	0.25	2.0	Midtones
0.95	0.95	0.0	Highlights
1.00	1.00	0.0	Clipping point

The shadow weight controls how much the under-exposure multiplier affects each region. The spline is interpolated to a 1024-entry LUT texture for GPU lookup.

4.3 Laplacian Pyramid Fusion

The fusion operates on a multi-scale Laplacian pyramid. The number of levels is determined by $\log(imageWidth) / \log(downScalePerLevel)$ (typically 4-6 levels). At each level, the algorithm:

- Decomposes the exposure-compensated image into Laplacian bands
- Computes per-pixel weights based on three factors:
 1. **Well-exposedness:** Gaussian PDF centered at target luminance (0.5), $\sigma = gaussSize$
 2. **Contrast:** Laplacian edge magnitude (3×3 kernel)
 3. **Saturation:** Per-channel standard deviation
- Weights are squared for sharper transitions
- Bottom-up reconstruction blends the Laplacian bands using the computed weights
- `blendMpy` increases at coarser levels for dehazing effect

4.4 Fusion Map Application

After pyramid reconstruction, a **fusion gain map** is generated. This is a per-pixel multiplier that maps input brightness to output brightness. It's applied back to the original Bayer data (before demosaicing), preserving the full raw information. The gain map uses bicubic interpolation and is blurred for smooth transitions.

4.5 Key LTM Parameters

Parameter	Default	Range	Description
baseExpose	1.0	0.1–5.0	Base exposure multiplier for pyramid input
targetLuma	0.5	0.0–1.0	Target luminance for well-exposedness weight
overExposeMpy	auto	0.0–2.0	Over-exposure gain multiplier (auto = compressor+1)
underExposeMpy	0.85	0.0–2.0	Under-exposure reduction multiplier
fusionExpoHighLimit	64.0	1.0–100	Maximum allowed fusion gain
fusionExpoLowLimit	0.0625	0.001–1.0	Minimum allowed fusion gain
fusionLaplaceFactorMin	0.001	0.001–0.1	Minimum Laplacian weight floor
fusionExpoFactorMin	0.01	0.001–0.1	Minimum exposure weight floor
gaussSize	4.0	1.0–10.0	Gaussian weighting kernel size
dehazing	0.2	0.0–1.0	Dehazing strength (blendMpy scaling)
noiseMax	0.05	0.0–1.0	Noise-based gain limiter
gammaKSearch	1.0	0.1–3.0	Gamma for underexposure position calc
downScalePerLevel	2.0	1.5–4.0	Pyramid downscale factor per level
overexposedUpperLimit	2.5	0.5–2.0	Upper limit for overexposed areas
softLowerLevel	0.0	0.0–1.0	Soft level threshold
softUpperLevel	0.1	0.0–1.0	Hard level threshold
useSymmetricExposureFork	false	bool	Symmetric exposure fork calc
curvePointsCount	5	3–10	Number of spline control points

5. Denoising — ESD3D2 + GuidedUpsample (Corrected)

Developer correction: Only two denoising shaders are actually active in the pipeline: `denoise/esd3d2` and `denoise/guidedupsample`. All other denoise classes (SmartNR, Bilateral, BilateralSeparable, Wavelet) exist in the source code but are **NOT in the active PostPipeline chain**. The active pipeline is much simpler.

5.1 ESD3D2 — Multi-Scale SNN Bilateral + Guided Upsample

Node: ESD3D2 (the ONLY active denoise node — handles ALL luma + chroma)

Shaders: `denoise/esd3d2.glsl` + `denoise/guidedupsample.glsl`

Algorithm: Multi-scale SNN bilateral denoise with guided filter upsampling

The ESD3D2 node runs a two-stage pipeline:

Stage A: Subsampled esd3d2 (color denoise at low resolution):

- Scale factor: `clamp(sqrt(noiseS×0.5 + noise0) / noiseTarget, 1, 4)`
- If scale > 1: downsample input, run esd3d2 at low res, guided upsample back
- If scale = 1: run esd3d2 at full res, then guided upsample

Stage B: Full-res esd3d2 (moire/chroma cleanup):

- Always runs on the result with moire strength parameter

esd3d2.glsl — SNN bilateral with chroma detection:

- For each pixel, examines KSIZE×KSIZE with 4-fold symmetry (SNN)
- Luma weight: $w = 1 - d^2 / (d^2 + sigY)$ — noise-model driven
- Chroma weight: $w_2 = 1 - d^2 / (d^2 + sigZ)$ — adds moire detection
- Separates luma (sigY) and chroma (sigZ=sigY+chromaNoise×moire) paths
- Output: `chroma_normalized × luma_brightness` with LUMA mixing

guidedupsample.glsl — Guided filter upsampling:

- Computes linear model (a,b) at low-res: `output = a×guide_luma + b`
- Uses 3×3 window, $\sigma=1.2$, with variance threshold (0.001) for stability
- Bilinearly interpolates (a,b) to full resolution
- Applies at full-res using noisy image as guide → recovers detail
- Median outlier check (4.5× MAD) prevents ringing artifacts

Dynamic kernel sizing (esd3d2):

```
kernelSize = sqrt((noiseS + noise0) / noiseTarget) × noiseToKernelSize + 1
MSIZE = clamp(minSize + kernelSize - (kernelSize % 2), minSize, maxSize)
scaleFactor = clamp(sqrt(noiseS×0.5 + noise0) / noiseTarget, 1, 4)
```

Parameter	Default	Range	Description
enable	true	bool	Enable ESD3D2 denoising
luma	0.8	0.0–2.0	Luma strength multiplier (LUMA in shader)
moire	1.5	0.0–5.0	Moire/chroma noise reduction (MOIRE in shader)
minSize	7	1–21	Minimum kernel size for esd3d2
maxSize	21	1–51	Maximum kernel size for esd3d2
noiseTarget	1/256	0–0.1	Noise level for scale factor + min kernel

Parameter	Default	Range	Description
noiseToKernelSize	24.0	0–50	Noise → kernel size scaling
useColorDenoising	true	bool	Run subsampled color denoise pass

5.2 Inactive Denoise Nodes (code exists, NOT in pipeline)

The following denoise classes exist in the decompiled source but are **NOT registered** in the TunableRegistry and do not execute in the current pipeline:

- **SmartNR** — bilateral guide filter + hybrid median chroma (bilateralguide.glsl)
- **Bilateral** — standard 2D bilateral (denoise/bilateral.glsl)
- **BilateralSeparable** — separable bilateral (denoise/bilateralsep.glsl)
- **Wavelet** — wavelet-domain processing (wavelet2.glsl)
- **ESD3D** — older ESD3D variant (denoise/esd3d.glsl)
- **ESD3DBayer / ESD3DBayerCS** — Bayer-domain variants

These are likely legacy code or alternative paths for different camera modes. The active denoise is exclusively ESD3D2.

5.3 Why Only 2 Shaders Matter

The developer confirmed that ESD3D2 handles **all luma and chroma denoising**. No other denoise node is active. The two shaders work as a complementary pair:

- **esd3d2.glsl** does the heavy lifting — SNN bilateral noise removal with separate luma/chroma paths
- **guidedupsample.glsl** recovers detail at full resolution using the original image as guide
- Together they form a multi-scale denoise: cheap at low-res, detail-preserving at full-res
- Changing guidedupsample can shift the overall look toward smoother rendering

Important: All shader defines (NOISES, NOISEO, MOIRE, LUMA, KERNELSIZE, MSIZE, INSIZE, SCALE) are **compile-time variables injected from Java**, not constants in the shader. Editing their default values in the .glsl files has **no effect** — they are overridden before shader compilation via #define injection.

5.4 Other Inactive Denoise Classes

The remaining denoise classes (Bilateral, BilateralSeparable, Wavelet, ESD3D, ESD3DBayer, ESD3DBayerCS) exist in the source code but are not registered in TunableRegistry and do not execute. They may be legacy code or alternative paths for different camera modes.

5.5 Bilateral & BilateralSeparable (INACTIVE)

Bilateral (INACTIVE): Standard 2D bilateral with spatial sigma=2.5. Not in active pipeline.

BilateralSeparable (INACTIVE): Separable bilateral, not in active pipeline.

Parameter	Default	Range	Description
enable	true	bool	Enable filter
kernelSize	15	5–31	Filter kernel size (odd)
spatialSigma	5.0	1–20	Spatial Gaussian sigma
intensityMultiplier	1.0	0.1–5.0	Noise-based intensity sigma multiplier

5.6 Known Issues (Developer Notes)

Issue: Shadow Noise After Tonemap

The developer identified that rounding/quantization noise appears in **deep shadows after tonemap**. The denoiser runs before tonemap, so it cannot clean up artifacts that tonemap creates by lifting shadows.

- Noise is concentrated in deep shadows only
- Deep dark regions appear 'clean' to the denoiser (low signal → low noise estimate)
- After tonemap lifts shadows, tiny noise/rounding gets amplified 10-100× and becomes visible
- The noise model $\text{sigY} = \text{NOISES} \times \text{luminance} + \text{NOISEO}$ gives near-zero sigma when luminance ≈ 0
- The denoiser is blind to the fact that tonemap will later magnify those pixels

Current Workaround Config:

- Adaptive noise model: OFF (using custom noise model from sensorspecific)
- Use color denoise: OFF (only final esd3d2 pass runs)

Potential Fixes:

- Add shadow-aware noise boost in esd3d2 shader — inflate sigY when luminance is very low
- Add post-tonemap shadow cleanup pass — targeted denoise on deep shadows only
- Add dithering before quantization — noise-shaped dithering in shadow range
- Adjust noise model to account for post-tonemap amplification

Issue: merge/noisehist Adaptive Noise Model

The developer noted that the adaptive noise model in merge/noisehist causes ESD3D to work incorrectly at high ISO. The noisehist shader computes a 2D histogram of brightness × variance using median-of-squared-differences, which may not produce accurate noise estimates at high ISO where noise is heavy.

- At high ISO, the 3×3 median variance estimate may be unstable
- The 64-bin variance histogram may saturate or compress at high noise levels
- With adaptive noise model OFF, custom sensorspecific noise model is used instead

Issue: Compile-Time Shader Defines

All shader `#define` values (NOISES, NOISEO, MOIRE, LUMA, KERNELSIZE, MSIZE, INSIZE, SCALE) are **injected from Java at shader compile time**. The default values in .glsl files are dead code — changing them has no effect. These are controlled via:

- @Tunable parameters on ESD3D2 node (luma, moire, minSize, maxSize, noiseTarget, noiseToKernelSize)
- Noise model values (noiseS, noiseO) from sensor characterization
- Runtime calculations (KERNELSIZE, MSIZE, SCALE computed from noise model)

Proposed Fix: Shadow Boost for ESD3D2

Based on developer feedback and noise model analysis, the following fix is proposed to address rounding noise in deep shadows that becomes visible after tone mapping lifts those areas.

Problem Summary

- Rounding/quantization noise appears in deep shadows after tonemap
- Denoiser runs before tonemap — cannot clean post-tonemap artifacts
- Deep shadows (luma ≈ 0) have tiny sigY → denoiser barely filters them
- Noise model: $\text{sigY} = \text{noiseS} \times \text{luminance} + \text{noiseO}$ → collapses to noiseO in shadows
- After tonemap lifts shadows 20-50×, residual noise becomes visible

Noise Model Computation (from decompiled Parameters.m5262d)

When adaptive noise model is OFF, the custom noise model from sensorspecific is used:

```
// Average NoiseModelA/B/C/D across all channels
dArr2[i] = average(NoiseModel[i][R], [G1], [G2], [B])

// noiseS = linear in ISO
noiseS = B_avg + A_avg * ISO

// noiseO = quadratic in ISO, scaled by digital gain
dgain = max(ISO / maxAnalogGain, 1.0)
noiseO = D_avg * dgain^2 + C_avg * ISO^2
```

Computed noise values (sensor_2, Xiaomi 15 Ultra):

ISO	noiseS	noiseO	sigY at luma=0.005
800	0.00216	0.00788	0.00789
1600	0.00431	0.0314	0.0314
3200	0.00862	0.125	0.125
6400	0.0172	0.501	0.501

Proposed Solution: Shadow Boost Parameter

Add a new @Tunable parameter **Shadow Boost** to ESD3D2 that inflates the noise estimate in deep shadows, causing the denoiser to be more aggressive where tonemap will later lift.

Property	Value
Title	Shadow Boost
Category	Denoise
Description	Boost denoising in deep shadows to prevent post-tonemap noise
Default	0.5
Min	0.0
Max	2.0
Step	0.01

Shader modification (esd3d2.glsl):

```
// After computing sigY from noise model:
float luma = dot(cin, vec3(0.25, 0.5, 0.25));
float shadowFactor = 1.0 + SHADOWBOOST * clamp(1.0 - luma * 2.0, 0.0, 1.0);
sigY *= shadowFactor;
```

Effect by luminance level:

Luminance	Effective sigY	Effect
0.00 (deep black)	$\text{sigY} \times (1 + \text{SHADOWBOOST})$	Maximum denoising
0.10	$\text{sigY} \times (1 + \text{SHADOWBOOST} \times 0.8)$	Heavy denoising
0.25	$\text{sigY} \times (1 + \text{SHADOWBOOST} \times 0.5)$	Moderate boost
0.40	$\text{sigY} \times (1 + \text{SHADOWBOOST} \times 0.2)$	Light boost
0.50+	$\text{sigY} \times 1.0$ (no boost)	Normal denoising

Usage workflow:

1. Set Shadow Boost to 0.5 (starting point)
2. Capture a scene with deep shadows at target ISO
3. Use the Shadows seekbar to lift shadows in the result
4. If rounding noise visible → increase Shadow Boost
5. If shadows look over-smoothed → decrease Shadow Boost

Advantages:

- Auto-adapts to noise model changes (uses noiseS/noiseO uniforms)
- Adjustable from app settings (no shader recompilation needed)
- Only affects deep shadows — midtones and highlights untouched
- Complements existing Shadows seekbar (denoise first, then lift)

Integration:

- Add @Tunable field to ESD3D2.java: `float shadowBoost = 0.5f`
- Pass as uniform: `m51941(SHADOWBOOST, shadowBoost)`
- Modify esd3d2.glsl to apply shadow factor after sigY computation
- Appears in Tunable Settings → Denoise section alongside Luma, Moire, etc.

6. Noise Model

All denoising stages are driven by a **per-channel noise model** that characterizes sensor noise as a function of signal level. The model follows the standard photon noise + read noise formulation:

6.1 Noise Model Equations

```
noise_variance(pixel) = noiseS × pixel_value + noise0

Where:
  noiseS = shot noise coefficient (signal-dependent, ∝ 1/ISO)
  noise0 = read noise offset (signal-independent, sensor property)

After multi-frame merge (N frames, 0.9 efficiency):
  effective_noiseS = (sensor_noiseS × analogGain) / (N × 0.9)
  effective_noise0 = (sensor_noise0 × analogGain) / (N × 0.9)

Where:
  analogGain = ISO / 100
  N = number of merged frames
  0.9 = merging efficiency factor
```

6.2 Per-Channel Noise

The noise model stores separate coefficients for R, G, and B channels. Green typically has lower noise (due to higher quantum efficiency in Bayer CFA). The noise coefficients are measured per-camera from the Camera2 API sensor characteristics.

Parameter	Description
noiseS[0]	Red channel shot noise coefficient
noiseS[1]	Green channel shot noise coefficient
noiseS[2]	Blue channel shot noise coefficient
noiseO[0]	Red channel read noise offset
noiseO[1]	Green channel read noise offset
noiseO[2]	Blue channel read noise offset
analogGain	ISO / SENSOR_MAX_ANALOG_SENSITIVITY
frameCount	Number of merged frames (×0.9 efficiency)

6.3 How Noise Drives Denoising

- **ESD3D**: $\text{sigY} = \text{NOISES} \times \text{luminance} + \text{NOISE0} \rightarrow$ kernel size scales with $\sqrt{\text{noise}}$
- **SmartNR**: $\text{lumaLevel} = \sqrt{\text{noiseS}[1]} \times \text{isoMultiplier}$
- **Bilateral**: $\text{sigY} = \sqrt{\text{noisefactor} \times \text{NOISES} + \text{NOISE0}}$
- **Fusion**: $\text{maxGain} = \text{noiseMax} / \sqrt{\text{noiseS} \times 0.5 + \text{noise0}}$ — prevents noise amplification

7. Demosaicing

The demosaic stage converts the Bayer pattern (RGGB, GRBG, GBRG, or BGGR) to full RGB. Photon Camera implements multiple demosaic algorithms selectable via settings. The default (**Demosaic3**) uses a multi-pass directional interpolation approach.

7.1 Demosaic3 Algorithm

- **Pass 1** (`demosaicp1.glsl`): Interpolate green channel at red/blue positions using directional gradients
- **Pass 2** (`demosaicp2.glsl`): Interpolate red at blue positions and blue at red positions using the interpolated green
- Edge-aware: uses horizontal and vertical gradient magnitudes to choose interpolation direction
- Anti-moire: gradient thresholding prevents interpolation across edges

Algorithm	Description
Demosaic	Basic bilinear (compatibility mode)
Demosaic2	Improved directional interpolation
Demosaic3	Multi-pass edge-aware (default, best quality)
DemosaicQUAD	For Quad Bayer sensors (2×2 binning)
DemosaicCompute	Compute shader variant
MonoDemosaic	Monochrome sensor (no CFA)
BinnedDemosaic	For binned raw output

8. Color Processing Pipeline

8.1 Auto White Balance (AWB)

AWB computes per-channel gains (R, G, B) from scene analysis. The gains are applied multiplicatively in the AWB node after demosaicing. The algorithm considers the scene illuminant and uses a gray-world assumption with highlight-aware weighting.

8.2 Chromatic Aberration Correction

The **ChromaticFlow** node corrects lateral chromatic aberration using a gradient-based flow field. It analyzes the per-channel gradients and shifts channels to align. This operates on the demosaiced RGB image.

8.3 Color Correction Matrix (CCM)

The **ColorD** node applies the Camera2 API's Color Correction Transform (CCT) — a 3×3 matrix that maps camera RGB to sRGB (or Display P3). The matrix is sensor-specific and provided by the camera HAL. Additionally, the node can adapt the color rendering based on scene analysis (saturation, hue shifts).

The CCM is stored in `Parameters.colorCorrectionTransform` as a 3×3 float array. The node also applies the **analog balance** — per-channel gains from the raw capture that ensure white balance at the sensor level.

9. Histogram Equalization & Global Tone Mapping (INACTIVE)

Note: Both Equalization and GlobalToneMapping exist in the source code but are **NOT in the active TunableRegistry**. They may be used in other camera modes or are legacy code.

9.1 Equalization (INACTIVE)

The **Equalization** node is a complex tone mapping operator that combines histogram equalization with polynomial tone mapping and optional LUT application. **Note:** The `mo5217d()` method failed to fully decompile (1007 instructions), so the following is reconstructed from partial decompilation and shader analysis.

Algorithm (reconstructed):

- Compute histogram of luminance channel
- Build equalization curve: map input luminance $\rightarrow$ equalized luminance
- Mix equalized with original based on brightness: shadows get more equalization
- Apply polynomial tone mapping curve: $\text{output} = a \times x^3 + b \times x^2 + c \times x + d$
- The polynomial preserves hue by sorting RGB channels, applying curve to min/max, rescaling middle
- Optional 3D LUT application (64×64×64 tricubic lookup)
- Black point subtraction: $\text{output} = (\text{output} - \text{blackVec}) / (1 - \text{blackVec})$

GLSL: `equalize.glsl` — Key tone mapping functions:

```
// Polynomial tonemap (preserves hue)
minmax = pow(minmax, 3) * toneMapCoeffs.x +
         pow(minmax, 2) * toneMapCoeffs.y +
         minmax * toneMapCoeffs.z +
         toneMapCoeffs.w;

// Sinusoidal tonemap (alternative)
float tonemapSin(float ch) {
    return 0.5 - 0.5 * cos(pow(ch, 1/1.5) * pi);
}
```

9.2 Global Tone Mapping

The **GlobalToneMapping** node applies a two-pass global tone mapping using a heavily downsampled luminance guide:

- Pass 1: Downsample input to 1/8, then to 1/8 again (1/64 total)
- Apply `globaltonemapping.glsl` with low-res luminance as guide
- Pass 2: Further downsample to 1/512 and apply again
- Strength parameter `str` (currently **0.0 = disabled** in default config)

Note: The global tone mapping shader file was not found in the decompiled resources. It may be generated at runtime or embedded differently. The strength parameter being 0.0 suggests this stage is currently a no-op in the default configuration.

10. Sharpening (Sharpen2 is active)

Three sharpening stages are applied sequentially, each using different kernel sizes and approaches:

Node	Description	Shader
Sharpen	Luminance-domain unsharp mask	sharpening/sharpen33.glsl or sharpen55.glsl
Sharpen2	Second pass with different kernel	sharpening/sharpen55d.glsl
CaptureSharpening	Capture-parameter-aware sharpening	capturesharpening.glsl
SharpenDual	Dual-domain sharpening (luma+chroma)	sharpening/sharpen33d.glsl

The sharpening strength is user-configurable via the settings UI (sharpness seekbar). The sharpening operates on the luminance channel to avoid color fringing artifacts. Edge-aware variants use the gradient magnitude to modulate sharpening strength.

11. Key GLSL Shader Reference

All shaders are in `assets/shaders/` within the APK. The following table lists all significant shaders with their purpose.

Shader Path	Purpose
<code>merge/merge00.glsl</code>	Normalize raw Bayer, apply exposure compensation
<code>merge/avermix.glsl</code>	Running average: $\text{mix}(\text{current}, \text{new}, 1/N)$
<code>merge/hotpixeldetect.glsl</code>	Hot pixel detection via 3×3 median comparison
<code>merge/hotpixelcorrect.glsl</code>	Replace hot pixels from averaged data
<code>ltm/exposebayer2.glsl</code>	Create multi-exposure pyramid inputs from Bayer
<code>ltm/fusionbayer3.glsl</code>	Laplacian pyramid fusion with 3-factor weighting
<code>ltm/fusionmap.glsl</code>	Apply fusion gain map to original data
<code>ltm/expose.glsl</code>	Simple exposure encoding (gamma sqrt)
<code>ltm/fusion.glsl</code>	Simple 2-exposure Laplacian fusion
<code>denoise/esd3d2.glsl</code>	SNN bilateral denoise with chroma detection
<code>denoise/bilateral.glsl</code>	Standard 2D bilateral filter
<code>denoise/bilateralsep.glsl</code>	Separable bilateral (V+H passes)
<code>bilateralguide.glsl</code>	Guided bilateral for luma denoising
<code>hybridmedianfiltercolor.glsl</code>	Hybrid median for chroma denoising
<code>noisedetection44.glsl</code>	Noise map generation (7×7 Gaussian)
<code>medianfilter.glsl</code>	Separable median filter
<code>reinterpolatecolors.glsl</code>	Color artifact correction post-denoise
<code>tofloat.glsl</code>	Raw uint16 $\rightarrow$ normalized float conversion
<code>demosaic/demosaicp1.glsl</code>	Demosaic pass 1: green interpolation
<code>demosaic/demosaicp2.glsl</code>	Demosaic pass 2: R/B interpolation
<code>equalize.glsl</code>	Histogram EQ + polynomial tonemap + LUT
<code>wavelet2.glsl</code>	Wavelet decomposition (Haar-like)
<code>waveletinv2.glsl</code>	Wavelet reconstruction with noise thresholding
<code>sharpening/sharpen33.glsl</code>	3×3 unsharp mask
<code>sharpening/sharpen55.glsl</code>	5×5 unsharp mask
<code>capturesharpening.glsl</code>	Capture-aware sharpening
<code>lenscorrection.glsl</code>	Lens distortion correction
<code>chromaticcomp.glsl</code>	Chromatic aberration compensation
<code>sagamerge.glsl</code>	SAGA merge algorithm
<code>shadowtexturing.glsl</code>	Shadow texture application

12. Tunable Parameter Reference

All parameters marked with `@Tunable` can be overridden at runtime via the `PhotonCameraTuning.ini` file in the app's internal storage. The format is: `NodeName_paramName = value`

12.1 Exposure Fusion Parameters

INI Key	Default	Description
ExposureFusionBayer2_baseExpose	1.0	Base exposure multiplier
ExposureFusionBayer2_targetLuma	0.5	Target luminance for fusion
ExposureFusionBayer2_overExposeMpy	auto	Over-exposure gain multiplier
ExposureFusionBayer2_underExposeMpy	0.85	Under-exposure reduction
ExposureFusionBayer2_fusionExpoHighLimit	64.0	Max fusion gain
ExposureFusionBayer2_fusionExpoLowLimit	0.0625	Min fusion gain
ExposureFusionBayer2_gaussSize	4.0	Gaussian kernel size
ExposureFusionBayer2_dehazing	0.2	Dehazing strength
ExposureFusionBayer2_noiseMax	0.05	Noise gain limiter
ExposureFusionBayer2_downScalePerLevel	2.0	Pyramid downscale factor

12.2 Denoising Parameters

INI Key	Default	Description
ESD3D2_enable	true	Enable ESD3D2 denoising
ESD3D2_luma	0.8	Luma strength (LUMA define)
ESD3D2_moire	1.5	Moire/chroma strength (MOIRE define)
ESD3D2_minSize	7	Min kernel size
ESD3D2_maxSize	21	Max kernel size
ESD3D2_noiseTarget	0.00390625	Noise→scale+kernel mapping
ESD3D2_noiseToKernelSize	24.0	Noise→kernel scaling
ESD3D2_useColorDenoising	true	Subsampled color denoise pass

12.3 Merge Parameters

INI Key	Default	Description
PyramidMerging_detectThr	1.5	Hot pixel detection threshold
PyramidMerging_noiseMpyHigh	3.0	Adaptive noise high
PyramidMerging_noiseMpyLow	0.333	Adaptive noise low
PyramidMerging_enableHotPixelCorrection	false	Enable correction
PyramidMerging_enableAdaptiveNoise	true	Adaptive noise model
PyramidMerging_MAX_HOT_PIXELS	65535	Max hot pixels buffer
PyramidMerging_MAX_REASONABLE_HOTPIXELS	2000	Pruning threshold

13. Adjustment Strategy Guide

Use this guide to identify which parameters to adjust based on observed image issues.

13.1 Problem → Solution Matrix

Problem	Fix 1	Fix 2	Fix 3
Shadow rounding after tonemap	Shadow noise boost in esd3d2	Post-tonemap denoise	Dithering
Shadows too noisy	ESD3D2.luma ↑ (0.8→1.2)	ESD3D2.moire ↑	Check noise model
Highlights blown	fusionExpoHighLimit ↑	overExposeMpy ↑	baseExpose ↓
Flat / washed out	targetLuma ↓	dehazing ↑	Equalization curve
Over-sharpened / halos	Sharpness seekbar ↓	CaptureSharpening off	
Color noise in shadows	ESD3D2.moire ↑ (1.5→3.0)	useColorDenoising=on	
Loss of detail (over-smooth)	ESD3D2.luma ↓	maxSize ↓	guidedupsample tuning
Dark image overall	baseExpose ↑	overExposeMpy ↑	underExposeMpy ↓
Too much contrast	dehazing ↓	fusionExpoHighLimit ↓	GlobalTM.str ↑
Chromatic aberration	ChromaticFlow auto		
Moiré patterns	ESD3D2.moire ↑↑	Check sensorspecific	
Motion blur in frames	Gyro sorting auto	Frame rejection ↑	

13.2 Quick Tuning Workflow

Step 1: Capture test images in the target scene (low light, high contrast, etc.)

Step 2: Identify the primary issue using the matrix above

Step 3: Create `PhotonCameraTuning.ini` with the relevant parameter overrides

Step 4: Place the `.ini` file in the app's internal storage

Step 5: Re-capture and compare

Step 6: For GLSL shader changes, recompile the APK (see Section 14)

14. Recompilation Guide

The APK can be recompiled after modifications using **Apktool**. This section covers the complete workflow from modification to signed APK.

14.1 Modification Approaches

What	How	Recompile Method	Notes
GLSL Shaders	Modify .glsl files in assets/shaders/	Apktool recompile	Fast, no Java needed
Smali Code	Modify .smali files in smali/	Apktool recompile	For Java-level changes
Runtime INI	Create PhotonCameraTuning.ini	No recompile	For @Tunable params only
Resources	Modify res/ (layouts, drawables)	Apktool recompile	UI changes

14.2 Apktool Recompilation Workflow

```
# 1. Decompile (already done)
java -jar apktool.jar d photoncamera-hdr-v4.apk -o apktool_out

# 2. Make modifications to shaders/smali/resources
# Edit files in apktool_out/assets/shaders/
# or apktool_out/smali/

# 3. Recompile
java -jar apktool.jar b apktool_out -o photoncamera-modified.apk

# 4. Sign the APK (required for installation)
keytool -genkey -v -keystore my-key.jks -keyalg RSA -keysize 2048 -validity 10000 -alias mykey
jarsigner -verbose -sigalg SHA256withRSA -digestalg SHA-256 \
  -keystore my-key.jks photoncamera-modified.apk mykey

# 5. Install
adb install photoncamera-modified.apk
```

14.3 Common Modifications

Increase shadow brightness in LTM:

```
# In assets/shaders/lrm/exposebayer2.glsl
# Change STRLOW and STRHIGH defines
# Or modify the baseExpose parameter in smali
```

Reduce denoising strength:

```
# In assets/shaders/denoise/esd3d2.glsl
# Change: #define LUMA 0.0 → #define LUMA 0.3
# Or reduce NOISES/NOISE0 multiplier
```

Change fusion target luminance:

```
# In assets/shaders/lrm/fusionbayer3.glsl
# Change: #define target 0.5 → 0.4 (darker target)
```

Analysis generated by decompilation of Photon Camera HDR v4 (com.particlesdevs.photoncamera.hdr v0.97). JADX 1.5.1 for Java source, Apktool 2.10.0 for smali/resources. Some methods failed to decompile (marked in document). GLSL shaders were fully extracted and analyzed.